



ACADEMIC PROJECT REPORT & VIVA DOSSIER TECH KARMA CLASSES



Full-Stack EdTech Web Application

A unified educational platform integrating CBSE Class 6-12 academic curricula, professional programming courses, automated test series, real-time presence telemetry, and cloud-optimized study materials.

Project Name: Tech Karma Classes (TechKarma)

Domain: Educational Technology (EdTech) / Full-Stack Web
Architecture

Purpose: College Project Submission & Viva
Defense

Developer: Sonu Kumar | Academic Session
2026

Executive Summary & Visual Mind Map

Tech Karma Classes is an end-to-end educational web application engineered to bridge standard school curricula (CBSE Classes 6 to 12) with industry-relevant computer science education (C, C++, Java, Web Dev, Python, AI/ML, Cloud, and Cybersecurity).



TECHKARMA SYSTEM ECOSYSTEM & ARCHITECTURE

Client Presentation (React 18)

- Vite 5 Lightning Fast HMR
- React Router DOM (SPA Routing)
- 15 Dynamic Themes (LocalStorage)
- Lucide React Modern Iconography
- Marked (Rich Markdown Parsing)

Application Logic (Express.js)

- Modular RESTful API Endpoints
- Stateless JWT Authentication
- Role-Based Access Control (RBAC)
- Multer File Upload Middleware
- Auto-Grading Examination Engine

Database Layer (MongoDB)

- Flexible Document Schema (Mongoose)
- User, Course, Material, Exam Collections
- Nested Arrays for Syllabus & MCQs
- Indexed Email & Phone Constraints
- BSON Native Binary Performance

Real-Time Telemetry (Socket.io)

- Bi-directional Full-Duplex WebSockets
- Multi-Tab Client De-duplication
- Live Student Online Presence
- Automatic Reconnection
- Resilience
- Broadcast Channel to Admin Panel

Automation & Cloud CDN

- Puppeteer Automated PDF Renderers
- PyMuPDF (150 DPI) Compression
- Hugging Face CDN Object Storage
- 1-Click High-Speed Downloads
- Zero Server Bandwidth Choke

Security & Ops (DevOps)

- bcryptjs (Salted Cryptographic Hash)
- CORS Cross-Origin Policy Protection
- Serverless Architecture (Vercel)
- Nodemon & Hot Module Reload
- Dotenv Environment Isolation

Primary Problem Addressed

Existing educational sites are either restricted purely to school boards or exclusively to tech courses. Students are forced to juggle between multiple platforms, deal with broken download links, and pay exorbitant fees. TechKarma unifies school and coding curricula into a single high-speed platform.

Key Technical Innovations

Offloaded 200MB+ PDF documents to a cloud storage CDN, automated exam scoring with zero server state, engineered multi-tab session deduplication via WebSockets, and implemented 15 instant-switching CSS variable themes.

Comprehensive Tech Stack & Justifications

Every technology, library, and runtime in this project was selected after evaluating performance benchmarks, bundle size impacts, and developer ergonomics:

TECHNOLOGY	ROLE / LAYER	ENGINEERING JUSTIFICATION ("WHY USED?")
React 18	Client UI Framework	Declarative component-based architecture allowed reusable modules (e.g. <code>McqViewer</code> , <code>ClassCard</code> , <code>Navbar</code>). Virtual DOM guarantees optimal re-rendering.
Vite 5	Build Tool & Dev Bundler	Replaced legacy Webpack/CRA. Employs native ES Modules (ESM) to deliver sub-second hot-module replacement (HMR) and optimized Rollup production builds.
React Router v6	SPA Client Routing	Enables fluid, zero-page-reload navigation across deep routes (<code>/notes</code> , <code>/courses</code> , <code>/admin</code>) with synchronized history restoration.
Node.js & Express	Server Runtime & REST API	Non-blocking asynchronous I/O event loop handles high-concurrency student queries with minimal memory footprint. Clean middleware pipelines for auth and logging.
MongoDB & Mongoose	NoSQL Document Store	Flexible JSON-like BSON documents naturally map to deeply nested academic structures (e.g. exams with questions and dynamic answer indices) without relational schema rigidity.
Socket.io	Real-Time WebSockets	Full-duplex persistent connection between client and server for live telemetry (online student count) with automatic fallback to long-polling when needed.
JWT & bcryptjs	Authentication & Hashing	bcrypt hashes passwords with cryptographic salt rounds. JWT ensures stateless authorization where user session is cryptographically signed and stored on client.
Multer	Multipart File Handling	Parses <code>multipart/form-data</code> file uploads directly on the server with mime-type validation to safely store administrative PDF notes.
Puppeteer (Chrome CDP)	Headless Automation	Automates headless Chromium instances to programmatically render styled HTML syllabus into high-precision, standard A4 formatted educational PDFs.
PyMuPDF (fitz)	Python Compression Engine	Downsamples embedded image DPI to 150 and re-encodes pages with 70% JPEG compression, slashing PDF file weights by 75–80% without losing text crispness.
Hugging Face Datasets	Cloud Object CDN	Hosts hundreds of megabytes of downloadable notes completely free of charge, avoiding GitHub repository

TECHNOLOGY	ROLE / LAYER	ENGINEERING JUSTIFICATION ("WHY USED?")
		bloat and Vercel 50MB serverless bundle limits.
CSS3 Custom Variables	Theming Engine	15 cohesive color palettes dynamically injected via data-theme attribute and synced to localStorage, eliminating bulky CSS frameworks.



Examiner Insight: If asked why MERN Stack was favored over traditional LAMP (Linux, Apache, MySQL, PHP) or Django/PostgreSQL, explain that *JavaScript ubiquity across both client and server reduces context switching*, and *MongoDB's document model represents JSON payloads with zero object-relational mapping impedance*.



Detailed REST API Specifications

METHOD	ENDPOINT PATH	ACCESS LEVEL	PURPOSE & REQUEST PAYLOAD
POST	/api/auth/register	Public	Registers student. Validates unique email/phone, hashes password with bcrypt.
POST	/api/auth/login	Public	Authenticates credentials, compares hash, signs & returns 30-day JWT.
GET	/api/auth/users	Admin Only	Fetches registered students telemetry and last-activity timestamps.
GET	/api/courses	Public	Retrieves summer courses and bootcamps with fees and highlights.
POST	/api/courses	Admin Only	Creates course record with syllabus, thumbnail, and pricing.
GET	/api/materials	Public	Filters study materials by subject, class grade, or content type.
POST	/api/materials	Admin Only	Accepts multipart/form-data to upload new documents via Multer.
GET	/api/exams	Public	Fetches test series according to subject ID and difficulty (Easy/Medium/Hard).
POST	/api/exams/:id/submit	Student Token	Validates answers against DB answers array, grades score, returns %.
POST	/api/queries	Public	Accepts admission/counseling inquiries from public prospective leads.

METHOD	ENDPOINT PATH	ACCESS LEVEL	PURPOSE & REQUEST PAYLOAD
GET	/api/stats	Public	Returns aggregated platform metrics (total registered students, page hits).

1. High-Speed PDF Material Vault

Mechanism: Instead of loading hefty files into local server memory, materials are mapped in `pdfManifest.json` and linked directly to an optimized Cloud CDN (Hugging Face Datasets).

```
<a href=
  {\`https://huggingface.co/datasets/.../${note.file}\`}
  target="_blank" rel="noreferrer" className="btn btn-
  primary"> <Download size={16} /> Download PDF </a>
```

Benefit: 0% server CPU strain, instantaneous parallel downloads, and zero risk of bandwidth bottlenecking.

2. Automated Online Exam Engine

Mechanism: Built as a 3-stage finite state machine: `setup` → `active` → `result`.

- Countdown timer managed via non-drifting `useEffect` intervals.
- Automatic trigger on timer expiration (forces instant submission).
- Anti-Cheat: Questions are displayed on client, but **answer verification is performed exclusively on the server**.

```
exam.questions.forEach((q,
  idx) => { totalMarks +=
  q.marks || 1; if
  (userAnswers[idx] ===
  q.correctOptionIndex)
  score += q.marks || 1; });
```

3. Real-Time Telemetry & Presence

Mechanism: Bi-directional WebSocket channels synchronize active user count to the Admin Dashboard.

- Maintains dual state maps: `socketUserMap` & `activeUsersMap`.
- **Multi-Tab De-duplication:** A user opening 5 browser tabs is counted as 1 distinct student using a `Set<socketId>`.
- Emits `livePresenceUpdate` on connect, identification, and disconnect events.

4. 15-Theme Dynamic Engine

Mechanism: Pure CSS variables configured on the `:root` element, controlled via custom React state and `localStorage`.

- Themes: Dark, Light, Cyberpunk, Luxury Gold, Dracula, etc.
- Zero FOUC (Flash of Unstyled Content) achieved by synchronizing attributes inside `useLayoutEffect` prior to screen paint.

5. Course Catalog & Admission Inquiry Pipeline

End-to-End Flow: A student browsing `Courses.jsx` evaluates discounted pricing (original vs. offer), course duration, and feature bullets. Clicking **"Enroll Now"** transitions the user to `Admission.jsx` with pre-filled course context. Upon submission, the record is committed to MongoDB (Query collection) and flagged as

Pending. The administrative team can inspect, triage, and mark inquiries as Resolved within AdminDashboard.jsx.

Case Studies in Technical Problem Solving



Problem 1: Hefty PDF File Sizes & Cloud Host Limits (>200 MB)

Issue: Handwritten scans and Puppeteer exports exceeded Git push limits (100MB) and Vercel serverless payload restrictions (50MB).

Resolution: Built a Python pipeline using PyMuPDF (`compress_all.py`) to render pages at 150 DPI and 70% JPEG quality, dropping sizes by 75%. Scripted `upload_all_to_hf.py` to offload storage to Hugging Face Cloud CDN.



Problem 2: Cross-Origin Resource Sharing (CORS) Blockade

Issue: Browser security blocked React (port 5173) from requesting Node (port 5000).

Resolution: Configured `cors()` middleware in Express and established a development proxy inside `vite.config.js` under `/api`.



Problem 3: Single Page Application (SPA) 404 on Direct Route Refresh

Issue: Visiting `/notes` directly on production returned 404 because Vercel sought a physical file.

Resolution: Formulated a URL rewrite manifest in `vercel.json` routing all wildcard paths (`/(.*)`) to `index.html`.

Top 6 Viva Questions & High-Impact Answers

Q1. Why did you choose MongoDB over a relational database like MySQL?

Answer: "Our educational domain involves heterogeneous, deeply nested data—such as dynamic MCQ options, variable course feature lists, and chapter syllabi. MongoDB's document model represents these JSON structures natively without requiring costly multi-table SQL joins."

Q2. How is authentication made stateless using JWT?

Answer: "Instead of persisting user session states inside server memory, the server signs a cryptographically verifiable JSON Web Token containing the user's ID and role. The client includes this token in the `Authorization: Bearer` header, which the backend verifies using a secret key without any database lookup."

Q3. What is the advantage of Socket.io over standard HTTP polling?

Answer: "HTTP polling requires continuous, repetitive request-response handshakes that consume bandwidth and CPU. Socket.io establishes a persistent, full-duplex TCP WebSocket connection, allowing the server to push real-time telemetry instantly with minimal latency."

Q4. How do you prevent cheating in your online examination engine?

Answer: "Although questions and options are served to the client, the correct answer indices are kept strictly confidential on the backend. When a student submits their choices, grading and mark tallying occur in the Express controller, preventing any inspect-element tampering."

Q5. How do you prevent ghost connection counts in Socket.io?

Answer: "We map each user ID to a Set of socket IDs. If a student opens three tabs, all three sockets belong to the same set. The student is marked offline only when the set becomes empty (i.e. every tab is closed)."

A step-by-step technical breakdown of how the brand identity was upgraded, mathematically pre-processed to remove raster checkerboard artifacts, integrated across all UI search bars, and transformed into an interactive 3-D physics-based element:

1. Algorithmic Background Extraction (Python)

The Problem: The raw uploaded emblem was an RGB JPEG containing a simulated checkerboard transparency grid. Placing it directly resulted in unsightly gray/white square artifacts.

The Algorithm: Engineered a *Breadth-First Search (BFS) Flood Fill* in Python using NumPy and PIL. Starting strictly from the outer boundaries, pixels were evaluated for chromatic grayness:

```
is_gray = (|R-G| < 18) & (|G-B| < 18)
& (R > 125) # Boundary BFS traverses
only outer background
arr[visited_mask, 3] = 0 # True Alpha
Channel
```

Protected the internal emblem pixels (white book pages, code brackets) while stripping 240,000+ exterior pixels into an authentic transparent 32-bit RGBA PNG.

2. Multi-Resolution Favicon & Cache-Busting

The Problem: Web browsers aggressively cache tab favicons inside internal memory caches, ignoring standard file replacements.

The Resolution:

- Generated mipmapped favicon.ico packaging 16x16, 32x32, 48x48, and 64x64 resolutions.
- Injected explicit HTTP cache-busting version query parameters into index.html:

```
<link rel="icon" sizes="32x32"
href="/logo.png?v=20261004b" /> <link
rel="shortcut icon"
href="/favicon.ico?v=20261004b" />
```

Guaranteed instant browser cache invalidation and instantaneous tab icon refreshes across Chrome, Edge, and mobile viewports.

3. Cross-Platform Search Bar Integration

Integrated the emblem directly into user interaction points:

- Study Material Vault (/notes):** Embedded a glowing 32px circular emblem within the search input with adjusted left padding (3.8rem).
- Admin Telemetry (/admin):** Injected miniature brand badges into Student & NCERT book search inputs.
- Footer & Login Card:** Replaced generic graduation cap SVGs with the high-resolution brand badge.

4. 3-D Interactive Flex & Mouse Physics

Dynamic Mouse Parallax: Implemented real-time cursor tracking in React:

```
const rotX = ((centerY - y) / centerY)
* 24; const rotY = ((x - centerX) /
centerX) * 24; // Transforms in 3D
perspective space: transform:
perspective(800px) rotateX(${rotX}deg)
rotateY(${rotY}deg) scale3d(1.18,
1.18, 1.18) translateZ(28px);
```

Paired with continuous idle floating keyframe physics (@keyframes logo3dFloat) and dynamic cyan/neon drop-shadow aura for an authentic 3D holographic feel.



Examiner Takeaway: Demonstrates practical proficiency in combining *computer graphics pre-processing* (Python/Pillow), *browser performance caching* (HTTP headers), and *modern CSS3 3D Matrix Mathematics* with React event-driven UI state.